

PaxosLease Revisited: A Checked Model of Diskless Distributed Leases

Márton Trencsényi (mtrencseni@gmail.com)

Abstract

PaxosLease is a protocol by which a quorum of acceptors grants time-bounded exclusive ownership with no durable acceptor lease state and no disk write on the lease acquisition path. This paper gives a precise, machine-checked statement of the protocol and of its standard use, electing a Multi-Paxos leader that recovers once per epoch and then commits with single-round appends. Three results are new. First, the restart quarantine an acceptor must observe after losing its volatile state is the proposer attempt duration, a bound tight in the checked model and machine-checked at the boundary in both directions. Second, timer placement is safety-critical: starting the attempt timer at prepare-quorum receipt can produce two simultaneous lease owners. Third, when a proposer abandons an acquisition attempt and retries, messages left over from the abandoned attempt can make acceptors report a lease the proposer itself appears to own; unless such a report counts as open only while the proposer is renewing that very lease, two simultaneous owners follow at any quarantine length. The lease transition system is formalized in TLA+ and checked by TLC, the timing arithmetic is proved in TLAPS, and an executable reference model encodes the same rules a second time.

1. Introduction

PaxosLease [1] is a protocol by which a quorum of acceptors grants time-bounded exclusive ownership, diskless in the sense that acceptors keep no durable lease state and the lease acquisition path writes nothing to disk. This paper gives a precise, machine-checked statement of the protocol and of its standard use, electing a Multi-Paxos leader that recovers once per epoch and then commits with single-round appends.

A lease is a lock whose ownership is bounded by time [3]: a process may act as owner only until its lease expires, and a process that granted the lease refuses conflicting grants until its own exclusion interval has expired. The time bound is what makes the lock survivable in a distributed system, where the remaining processes cannot tell a failed owner from a slow one, and it is also what makes lease safety delicate, since exclusion, the property that at any real instant at most one process has the authority to act as owner, then depends not only on messages and quorums but on assumptions about elapsed time. Clocks need not be synchronized, but the intervals measured by different processes must have bounded error. PaxosLease uses ballots and intersecting quorums, as Paxos does, to grant such a time-bounded leadership right; it does not replace Paxos's own recovery (Section 5).

The paper contributes the following. First, the tight restart bound: the quarantine an acceptor must observe after losing its volatile state is the proposer attempt duration D_p , tight in the checked model, checked at the boundary in both directions, smaller than the acceptor exclusion duration and smaller than the maximal lease time (Section 7). Second, the validity conditions of attempt evidence, each backed by a machine-found safety-violating two-owner ex-

ecution: the attempt time bound, whose placement admits a four-way design space of repairs and a tempting unsafe fifth choice, and the renewal qualifier, that a proposer's own reported lease is an open response only while renewing that very lease, a condition the model checker itself surfaced when retry was modeled explicitly (Section 8). Third, the implementation audit: eight source-level findings in Keyspace and ScalienDB read against the checked model, backed by checked projections of the shipped protocol and by executable witnesses (Section 11, Appendix C). Supporting these: a statement of the algorithm precise enough to resolve the internal inconsistency of the previous publication, each step bound to the TLA+ action that formalizes it, the checked module being normative (Section 4); the leased-leader Paxos lifecycle stated in the same discipline (Section 5); and a repository in which the negative claims are not just prose: mechanically generated one-rule-weakened variants whose violations TLC must find on every run, fifteen structured witnesses, an executable reference model that encodes the rules a second time, a fail-fast evidence pipeline, and a single-file runnable demonstration (Sections 9 and 10).

The 2012 publication states the safety-critical timer rule inconsistently, one way in its figure and another in its pseudocode. The two production implementations, Keyspace and ScalienDB, copied the pseudocode, were protected by unintended implementation choices, but are defeated by ordinary timer-dispatch latency. Section 11 tells that story as engineering lessons, and Appendix B and Appendix C carry the forensics. This paper supersedes and replaces it: Section 2 gives the protocol in plain terms and Section 4 gives it precisely, and a reader who wants to implement PaxosLease should work from this new source and

not from [1].

The rest of the paper is organized as follows. Section 2 describes the protocol in plain terms, Section 3 states the system model, Section 4 states the algorithm, and Section 5 states the leased-leader algorithm. Section 6 gives the safety argument, Section 7 establishes the quarantine bound, and Section 8 maps the design space of the attempt time bound. Section 9 describes the checked models and tests, Section 10 presents the runnable demonstration, Section 11 audits the unformalized years, Section 12 draws the lessons, and Section 13 discusses related work. Section 14 lists limitations. Appendix A records the checked invariants, Appendix B details the inconsistency in the original statement, and Appendix C details the implementation audit, and Appendix E records how the repository was built.

2. How PaxosLease Works

PaxosLease grants one process the exclusive right to act as owner of something, for a bounded stretch of time, using a set of machines any of which may crash and none of which writes to disk in order to grant it. This section describes how it does that in plain terms. It is intended for a reader who has not seen the protocol before and who should not have to reconstruct it from the numbered steps of Section 4.

The reason disklessness is the interesting constraint is that the obvious way to grant an exclusive right is to write down who has it. A grantor that has written nothing down and then restarts remembers nothing, and a grantor that remembers nothing will happily grant the same right twice. Every difficulty in this protocol descends from that sentence.

The idea is to run Paxos, with one substitution: the value being agreed upon is not a log entry but a statement of the form “node 3 owns the lease, for the next seven seconds.” Paxos guarantees that a quorum of acceptors agrees on at most one value, which is exactly the guarantee a lease needs. The substitution earns something Paxos does not have, which is that the agreed value stops mattering on its own. An acceptor does not need to remember a lease forever, only until it expires, and that is what makes it safe for the acceptors to keep the whole thing in memory.

An acquisition is two rounds. A proposer picks a ballot number that nobody has used before and sends it to every acceptor, asking two things at once: to promise not to entertain any smaller ballot from now on, and to respond whether it is currently holding a live lease for another owner. If a majority of acceptors answer, and none of them reports a live lease belonging to another owner, the proposer sends a second message asking those acceptors to record it as the owner for a fixed duration. When a majority confirm, the proposer is the owner, and it informs the learners.

Majorities are what make this work, because any two majorities of the same set share at least one member. A

second proposer trying to acquire the lease cannot avoid talking to at least one acceptor that already answered the first, and that acceptor either reports the live lease, which stops the second proposer, or has promised a ballot that makes the first proposer’s remaining messages ineffective. There is no configuration of messages in which two proposers each collect a majority in ignorance of the other.

What Paxos does not have to reason about, and this protocol does, is time. Two obligations follow, and they are the subject of much of this paper. The first is that an owner’s authority must end before the acceptors’ exclusion does. The acceptors are holding the lease open for a fixed duration measured on their own clocks, and if the owner believes it holds the lease for even slightly longer than the acceptors are willing to enforce it, then in that gap a second proposer can acquire while the first still believes itself the owner. The second is about forgetting: An acceptor that crashes loses its promises and its record of the live lease, and if it restarts and immediately starts answering again, it can help a second proposer acquire a lease that, as far as everybody else is concerned, is already held. The repair is a quarantine: a restarted acceptor refuses all lease-layer messages for a fixed interval, and the design question is how long that interval must be.

Two smaller pieces complete the picture. A lease is renewed by running the same two rounds again, under a higher ballot, before the current one expires; a renewal that fails to complete extends nothing, and the owner’s authority simply runs out. A lease may also be released early, which is a courtesy that speeds up the next acquisition and is never required for safety.

That is the protocol. Everything after this section is either a precise statement of it, a proof obligation about it, or an account of the ways it can be got wrong. The precise statement is Section 4, whose numbered steps are bound to a machine-checked TLA+ module.

3. System Model

The system contains a finite set of proposers and a finite set of acceptors. A quorum system is a collection of sets of acceptors, any two of which intersect; the executable models use majorities.

Processes are not Byzantine, but any process may crash. A crashed proposer loses its lease authority, and a crashed acceptor loses its PaxosLease state, including its promised ballot and its accepted lease. This is the sense in which the protocol is diskless. When PaxosLease is used with Paxos, the volatile loss does not extend to the Paxos log, whose acceptor state remains durable.

One durability caveat applies to proposers. Ballots must be globally unique, including across restarts of the same proposer. Uniqueness is obtained from a restart counter written to stable storage [1]. (A sufficiently large random component gives probabilistic rather than deterministic uniqueness, and a deployment that relies on it should

say so.) The counter is one word written per proposer restart, not a write on the protocol path, but a fully diskless deployment must obtain uniqueness in some other way. Flea [10], for example, derives ballots from loosely synchronized clocks (Section 13). The models make uniqueness structural: the TLA+ specification tracks used ballots in a global history set, and the Python models use the counter-major (*counter, restart, node*) layout of Section 4, so ballots remain comparable across proposers. Note what the restart component provides: uniqueness across a crash, not ordering, since the counter restarts under a higher restart component and $(1, restart+1, node) < (100, restart, node)$.

Messages may be delayed, reordered, duplicated, or lost. A message carries a ballot and, in an accept request, the identity of the proposed lease instance. Messages do not carry synchronized timestamps. Each process uses its own elapsed-time measurements to decide whether a lease or a quarantine interval has expired. Messages are not tied to connections: unless stated otherwise, a message sent before its addressee crashed may be delivered after the restart. This clause is not a technicality. One of the three headline results depends on it, and the checked models test it explicitly against the connection-oriented alternative, in which a crash discards the messages addressed to the crashed process (Sections 8 and 9).

The timing condition needed for safety is a containment condition. Whenever acceptor *a* accepts a lease for proposer *p*, the end of *p*'s usable interval must come no later than the end of *a*'s exclusion interval. If all clocks run at the same rate and the proposer starts its timer before the acceptor starts its own, then equal local durations suffice. If clock rates can differ by at most ρ , then a conservative choice is

$$D_P \leq \gamma D_A, \quad \gamma = \frac{1 - \rho}{1 + \rho},$$

where D_P is the proposer's local duration, D_A is the acceptor's local exclusion duration, and $\gamma \leq 1$ is the worst-case ratio at which the proposer's clock may run fast while the acceptor's runs slow. The arithmetic content of this condition, that bounded rates together with the cross-multiplied duration inequality imply real-time containment, is discharged by the TLAPS obligation `DriftContainmentArithmetic`.

When the proposer's timer starts is part of the protocol, not a detail of implementation. The model starts the pending deadline when `Prepare` is sent. Section 8 shows that the alternative, starting it at prepare-quorum receipt, is unsafe when promises are volatile, and Section 7 shows that the prepare-time rule is also what makes a small quarantine possible.

4. PaxosLease Algorithm

We state the algorithm as numbered steps. Every step names, in brackets, the action of

`tla/spec/PaxosLease.tla` that formalizes it. The module, not this prose, is the normative statement: TLC re-checks it on every run of the evidence pipeline.

4.1. State

Proposer state is per attempt and volatile, with one durable exception.

- the current ballot b , and the phase of the attempt (idle, preparing, accepting);
- the pending deadline d of the attempt, assigned when `Prepare` is broadcast (T1) and stored, not scheduled;
- the renewal base: the ballot of the lease this attempt renews, or none if the attempt is a fresh acquisition (P1, P2);
- the identities of the acceptors that have responded in the current attempt, kept as a set rather than a count (P2);
- when active, the authority deadline and the quorum certificate that established the lease;
- the restart counter, which is the one piece of proposer state written to stable storage (Section 3).

Acceptor state is entirely volatile, which is what makes the protocol diskless and what the restart quarantine exists to repair.

- the highest ballot promised in the current volatile epoch;
- the accepted lease instance, if one exists;
- the local deadline until which that lease excludes conflicting grants;
- after a restart, the deadline until which the acceptor refuses all lease-layer messages (A4, T3).

A lease instance is identified by (*owner, ballot*). Ballots are globally unique, so the owner is defensive redundancy, but the pair is what release must match (A3).

4.2. Proposer

P1 [StartAcquire]: Choose a fresh ballot b , never used by any proposer, including this proposer's own earlier incarnations: counter-major (*counter, restart, node*), where *counter* increments per attempt and *restart* is a durably stored counter incremented at every process start, so no ballot is reused across a crash. (Fresh means unique, not maximal: after a restart the counter begins again, and the new ballots may compare below promises the proposer obtained before crashing. That is safe, acceptors simply reject them, but the proposer stalls until the catch-up rule of T4 lifts its counter.) Assign the pending deadline $d := now + D_P$ as stored state, and broadcast `Prepare(b)`.

P2 [DeliverPromise]: On a promise for the current b : if it reports no live lease, record the responder's *identity*. For a renewal attempt, a response is also compatible, but only when it reports the same owner *and the same ballot* as the active lease captured at P1 as the attempt's renewal base. Quorums are sets of distinct acceptors, not counts of messages (Section 11, fourth finding). The renewal clause is a check on where the reported lease came from, not merely on who owns it: the attempt records at P1 which lease it is renewing, and P2 admits a self-owned report only if it names that exact lease. A self-owned record under any other ballot may have been installed by an accept request an abandoned attempt left in flight, and it blocks like a foreign lease (Section 8).

P3 [SendAccept]: When promises from a quorum are recorded and $now < d$: broadcast `Accept(b, self)`.

P4 [DeliverAccepted]: On accepted responses for b from a quorum of distinct acceptors, and only if $now < d$, checked against the stored deadline inside this transition rather than by any timer callback: become active until d , record the certificate, and broadcast `LearnChosen(self, d)`.

P5 [AbandonAttempt, then P1]: An attempt that cannot proceed is abandoned: the proposer returns to idle, alive and keeping whatever lease it already holds, and retries with a fresh, higher ballot from P1. Messages of the abandoned attempt may still be in flight, and the ballot comparison in P2 and P4 discards every response belonging to it. Retry is not crash and restart, which strips authority and advances the durable restart counter; the model contains both transitions, and a dedicated configuration checks proposers abandoning and retrying across acceptor crash and restart at the quarantine boundary, with the abandoned attempt's messages still in the network (Section 9). The implementation projection of Section 11 additionally models the shipped retry timeout.

P6 [Release, optional]: To release, first stop acting as owner, then broadcast `Release(b)` naming the exact instance. Release is an availability optimization, never a safety requirement.

Renewal is a new acquisition by the same owner under a higher ballot, entered at P1. The ballot P1 records is the attempt's *renewal base*: a fresh acquisition has none, and a renewal has exactly the ballot of the lease it intends to extend. P2 tests self-owned reports against it. The old lease remains the only authority until the new accept quorum is complete, and a failed renewal extends nothing. Capturing the base at P1 rather than inferring it at each response means a fresh attempt can never be reclassified as a renewal by ownership changing while responses arrive.

4.3. Acceptor

A1 [DeliverPrepare]: On `Prepare(b)`: if $b \geq$ the promised ballot, set $promised := b$ and reply with the currently live accepted lease, reporting an expired lease as no

lease; otherwise reply rejecting, or not at all. A rejection is a liveness aid, and to serve T4 it should carry the promised ballot. Exclusion does not depend on rejections at all: an acceptor that stays silent refuses just as effectively, and the proposer's own deadline ends the attempt either way.

A2 [DeliverAcceptReq]: On `Accept(b, o)`: if $b \geq$ promised, set $promised := b$, record the lease (o, b) with exclusion deadline $now + D_A$, and reply accepted; otherwise reject.

A3 [DeliverRelease]: On `Release` from o with ballot b : clear the accepted lease only if it is exactly (o, b) . Matching on owner alone erases a later renewal (Section 9, 36-state trace).

A4 [RestartAcceptor]: On restart after losing volatile state: refuse every lease-layer message, prepare, accept, and release alike, until $now + Quarantine$.

4.4. Learner

L1: Activation is made application-visible through learners. The owner's own learner installs the absolute deadline d ; a remote learner may cache the owner with a conservative local expiry, but only the owner acts on the lease. A remote learner that measures the duration from the moment the message arrives, rather than from the moment the lease began, is assuming a bound on delivery delay, and that bound must then be stated (Section 11, sixth finding).

4.5. Timing rules

The timing rules are stated as named rules, not as properties implied by the steps.

T1 (attempt time bound): The pending deadline d is assigned when `Prepare` is broadcast (P1) and checked in the transitions that use the attempt's evidence (P3, P4). It is data, not a scheduled callback: dispatch order does not survive a pause, and the deadline must (Section 11, third finding).

T2 (containment): $D_P \leq D_A$, so an active owner's authority interval is contained in every certificate acceptor's exclusion interval. With clock-rate error bounds r_{min}, r_{max} the condition is $D_P \cdot r_{max} \leq D_A \cdot r_{min}$ (Section 3).

T3 (quarantine):

$$Quarantine \geq D_P,$$

rate-adjusted like T2. It is not the acceptor exclusion duration and it is not the maximal lease time; Section 7 derives the bound, shows by exhaustive checking that it suffices even when $D_P < D_A$, and shows that it is tight.

T4 (ballot hygiene): Safety requires exactly one property: ballots are globally unique and never reused across restarts (the durable *restart* component of P1). The rest of the rule is liveness. Because uniqueness does not order a restarted proposer's ballots above its pre-crash promises (P1), a proposer raises its counter above any ballot it observes in responses, which in turn requires rejections that carry the promised ballot (A1).

Two further rules earned their place through the audit rather than through the protocol: count quorums by acceptor identity or document and preserve at-most-once delivery per response (P2), and read the clock once per handler, comparing before subtracting, so that expiry checks are not exposed to time-of-check races or unsigned wraparound (Section 11, eighth finding).

The model has a variable `now`, and executable guards such as `now < pendingDeadline` read it directly. Since this can look like an assumption of synchronized clocks, we state precisely what it means. The variable `now` is the common real-time axis on which safety is asserted, and each local timer is a duration that has been conservatively translated onto that axis using the rate bounds below. When a process in the model compares `now` with its own deadline, it represents an implementation reading its own monotonic elapsed-time counter, never another process's clock. Similarly, the certificate variables `cert` and `certDeadline` that appear in the invariants are history variables. They record, for the benefit of the checked predicates, which acceptors granted the current lease and the exclusion deadlines those acceptors reported. The protocol itself never compares a deadline measured on one clock with a deadline measured on another.

5. Leased-Leader Algorithm

PaxosLease is not Paxos, and the two should not be confused. Paxos solves agreement for log slots, choosing at most one value in each slot [2]. Unlike the lease layer, its acceptors are durable: a promise and an accepted value must each reach stable storage before the acceptor replies, so committing one value in plain Paxos costs two round trips and two durable writes at every acceptor.

The leader optimization removes half of that. A proposer runs Phase 1 once under a single ballot for the whole log rather than for one slot, and the promises it collects reserve that ballot for every slot it will later fill. From then on it appends by running Phase 2 alone, so each new value costs one round trip and one durable write per acceptor instead of two. Phase 1 is amortized, not skipped, and the reservation lasts until another proposer's own Phase 1 wins a quorum of promises under a higher ballot.

This is what the lease contributes. A proposer holding a PaxosLease renews it, and may go on renewing it indefinitely: until it crashes, until the network prevents a renewal, or until the application hands leadership elsewhere. While it holds the lease no other proposer can acquire the lease, and hence no other proposer will run a Prepare phase, so the single reservation stays valid across arbitrarily many appends and the two round trips of the initial full round are amortized over all of them. Driven to the limit, the amortized cost of committing a value is one round trip and one durable write at each acceptor, which is what makes leased Multi-Paxos a practical way to implement a replicated log.

What the lease does not do is replace Phase 1. A proposer that has just acquired the lease does not know which values earlier leaders may already have had accepted, or chosen, and it must run a full first round to discover them and finish them before proposing anything of its own.

The standard use of the lease, and the one Keyspace and ScalienDB implement, is this classic Multi-Paxos leader optimization. We state the leader lifecycle precisely, in the discipline of Section 4: each step names its counterpart in the composition model (`tla/spec/PaxosLeasePaxos.tla`) and in the executable Python model.

M1 [AcquireLease; `acquire_lease`]: Win the PaxosLease of Section 4. This starts a fresh leadership epoch, and it *clears* any recovery standing from an earlier epoch.

M2 [StartRecovery/FinishRecovery; **Phase 1 of** `recover_and_become_ready`]: Run a log-wide Paxos Phase 1 under a fresh ballot. In the Multi-Paxos organization assumed here, and implemented by both audited systems, a promise quantifies over all instances rather than one [2, 9], so the single round both reserves the ballot for the entire log and discovers the accepted values that constrain recovery.

M3 [**repair**; **Phase 2 loop of** `recover_and_become_ready`]: For every slot that Phase 1 reports constrained, select the value attached to the highest-ballot accepted response for that slot, re-propose it under the reserved ballot, and learn the results. Phase 1 alone is not cleanup: a value that is discovered but not re-proposed leaves recovery to be finished by whatever proposal next touches the slot.

M4 [BecomeReady]: Only now is the leader ready. Readiness is per epoch: the invariant `ReadyImpliesRecovered` checks that a re-elected leader repeats M2 and M3 rather than relying on a previous epoch's recovery.

M5 [AdmitOrApply; `append`]: Fast mode. Each client command is placed, unaltered, into a genuinely new slot with a single accept round under the reserved ballot and no per-instance Prepare.

M1 relies on the exclusion theorem that Section 6 establishes for the lease layer; nothing else below does. Two facts about this optimization must be kept separate. Its safety is Paxos's alone: acceptors reject lower-ballot accepts regardless of any clock, so even two processes that simultaneously believe themselves leader cannot choose conflicting values, and skipping Prepare in M5 is sound because M2 already promised the whole log. The lease adds no safety to this and is needed for none of it. What the lease adds is efficiency and liveness, since with at most one real-time leader the fast path is not repeatedly preempted by dueling ballots, and local admission and lease-guarded reads become possible. One caution on reads: the lease alone does not make a local read current. A leader serving reads locally needs the lease, this epoch's completed recovery (M2, M3), and an applied state machine covering the relevant prefix; ownership without recovery reads stale

state. Conversely, the fast mode imposes only one obligation on the lease layer: no admission before the leader's own epoch has completed M3.

The separation is worth stating as a property of the code rather than as an intuition, because it is what makes the two layers independently reviewable. No Paxos transition reads PaxosLease state. An acceptor deciding whether to promise or to accept consults its own promised ballot and its own accepted values, and nothing else; it does not know whether the proposer addressing it holds the lease, and it would behave identically if the lease layer were deleted and leaders were chosen by a coin. No Paxos transition reads a clock. Deadlines, durations, drift bounds and the quarantine all live in the lease layer, and agreement on the log is a function of the message history alone. The coupling runs one way and touches one thing: the lease decides which proposer runs M2 through M5, and when it may begin. Put another way, the only part of Paxos this optimization modifies is the proposer, which skips the prepare phase while the lease condition holds. The acceptor and the learner are the unmodified Paxos ones.

6. Safety Argument

Theorem. At any time, at most one proposer is active.

The theorem is not a claim made only in prose. It is the invariant the checked module carries, written in `tla/spec/PaxosLease.tla` as

```
LeaseExclusivity ==
  Cardinality(active) <= 1
```

where `active` is the set of proposers that currently hold lease authority. Every configuration of Section 9 checks it, and the counterexample variants exist to violate it.

What follows is an argument rather than a mechanized proof of the transition system, and the distinction matters. TLC establishes the invariant exhaustively for the finite configurations it is given, and no further. The arithmetic the argument leans on is discharged mechanically as TLAPS theorems: timer containment, the coverage of forgotten facts by the quarantine, and the clock-rate condition of Section 3. Each clause of the argument below is also paired with a configuration or a weakened variant in which the model checker exhibits the two-owner execution that follows from dropping that clause, which is the sense in which the argument is falsifiable. A parameterized proof of the theorem itself, for all quorum systems and all cluster sizes, is not attempted here; Section 14 states what that omission leaves open. One asymmetry in that gap is worth naming. The argument below counts no acceptors: it uses exactly one property of the quorum system, that any two quorums share a member, which holds for majorities of every size. The acceptor count is therefore the dimension in which the argument is already general and the checking merely confirms it, and the three-acceptor runs support that reading, since they find the same violations in fewer states

than the two-acceptor runs rather than more. The dimensions in which finite checking is doing the real work are the number of proposers and the number of ballots, where the configurations are small and the argument has no comparable parametric step.

Argument. A proposer is active only if it holds a certificate from a quorum of acceptors. Suppose that proposer p is active with certificate quorum Q_p and that a different proposer q later becomes active. Before q can become active it must obtain a prepare quorum Q_q , and since quorums intersect, some acceptor belongs to both Q_p and Q_q .

If that acceptor still holds a live accepted lease for p when it answers q 's prepare, then q cannot proceed with a conflicting lease. If it answers with no live lease, then its exclusion interval for p has expired, and by timer containment p 's usable interval has expired as well, so p is no longer active. The remaining cases are lower-ballot races and acceptor crashes. Monotone promises dispose of lower-ballot races within a volatile epoch, while quarantine and the attempt time bound dispose of state loss across epochs. Both are necessary, and Sections 7 and 8 show what fails when either is weakened.

The intersection step needs one further clause. The record at the intersecting acceptor is not the earlier owner's authority. It is only that acceptor's in-memory record of having granted it, and the two can come apart: a delayed accept request from an abandoned attempt can overwrite the record while the owner goes on holding the lease, unaware that anything at the acceptor has changed. What a later acquirer may count is therefore constrained by provenance. A fresh acquisition may count only responses that report no live lease, and a response reporting the acquirer's own lease is compatible only with a renewal: an attempt begun while the acquirer is active, and only if the reported instance is the very lease being renewed. Under that rule, an overwritten record can mislead no one, since it either names a foreign owner, which blocks, or names the acquirer under a ballot that is not its renewal base, which blocks equally. The TLA+ module tests the proposer's activity and active ballot at response delivery; with a single attempt in flight and activation updating atomically, that coincides with the renewal base captured at P1 when the attempt begins, and the captured form is what an implementation should store and what the executable models do store, since separate callbacks and delayed learners can otherwise reclassify an attempt while its responses arrive. An alternative places the clause at the acceptor instead, as an invariant of the grantor: an acceptor never replaces a live lease of a different owner. Section 8 exhibits the two-owner execution that follows when the clause is dropped, and compares the two repairs.

The exclusion argument rests on the invariants listed in [Appendix A](#), of which the most important is timer containment: for every active proposer, the proposer's deadline comes no later than the exclusion deadline recorded by each acceptor in its certificate. Ballots and quorum intersection ensure that a later acquisition sees the earlier one,

and timer containment explains why an acceptor that no longer sees the earlier lease is not concealing a still-active owner.

Those invariants, eight in the lease layer and all checked by TLC on every run, divide into four groups by what they protect. Quorum-certificate invariants state that active authority comes only from distinct acceptors. Ballot invariants state that an acceptor that has promised a higher ballot cannot later accept a lower one, and that a release names the exact instance it clears. Timing invariants state that owner authority is contained in acceptor exclusion and that expired authority is not active. Restart invariants state that a quarantined acceptor holds no volatile lease state and cannot participate. We keep the groups separate in the implementation contract because they fail for different engineering reasons. Duplicate response accounting, stale cleanup messages, unsafe clocks, and immediate restart are different bugs, even though each can surface as a violation of lease exclusivity.

This is a safety argument only. It does not assert that any proposer eventually obtains a lease. Liveness requires eventual message delivery, sufficiently accurate timers, and a period during which a quorum of acceptors stays available. As in Paxos, two proposers can also starve one another with ever higher ballots, which implementations prevent with randomized backoff [1].

7. Restart Quarantine Bound

A restarted acceptor has forgotten two kinds of facts, promises and accepted leases, and we must determine how long it should stay silent so that neither can matter. The answer turns out to be the same for both, and it is smaller than one might expect.

A forgotten promise belongs to an attempt whose Prepare was sent at some time t_{prep} no later than the crash. Under the prepare-time timer rule, that attempt cannot activate after $t_{prep} + D_P$.

A forgotten accepted lease appears to be the harder case, because the acceptor promised exclusion for $D_A \geq D_P$, and an earlier draft of this paper accordingly claimed the bound $\max(D_P, D_A)$. That claim is too conservative, and the gap is instructive. Once the acceptor has forgotten the exclusion, the safety-relevant question is not whether the old exclusion interval has ended. It is whether any proposer that relied on the exclusion can still be, or can still become, active. The lease the acceptor forgot was accepted at some t_{acc} no later than the crash, on behalf of an attempt whose Prepare was sent at $t_{prep} \leq t_{acc}$, so the proposer it supports is not active past $t_{prep} + D_P$, regardless of D_A . Both forgotten facts therefore reduce to a single statement: everything an acceptor can forget belongs to an attempt that expires D_P after a Prepare that preceded the crash. That statement is the TLAPS obligation `QuarantineCoversForgottenFacts`. Hence, in the

discrete model,

$$Quarantine \geq D_P$$

suffices, and the acceptor exclusion duration does not appear in the bound at all.

The bound above is stated in the idealized time of the model, where every clock advances at the same rate. Real clocks do not, and the adjustment is the same one Section 3 applies to containment. The attempt is measured on the proposer's clock, which may run slow and stretch D_P out in real time; the quarantine is measured on the restarted acceptor's clock, which may run fast and use its interval up early. Taking the worst of both with the rate bounds $r_{min} = 1 - \rho$ and $r_{max} = 1 + \rho$ gives

$$Quarantine \geq \frac{D_P}{\gamma}, \quad \gamma = \frac{1 - \rho}{1 + \rho},$$

which is the containment condition $D_P \leq \gamma D_A$ turned around: the same γ deflates the acceptor duration you may rely on and inflates the quarantine you must serve. Both follow from the single TLAPS obligation `DriftContainmentArithmetic`, once with the acceptor duration and once with the quarantine in its place, and the Python timing module computes both bounds this way.

A word on the strength of this claim. The bound is the tight worst-case bound under the protocol and failure model stated here, established by the arithmetic lemmas together with finite model checking rather than by a parameterized proof. What tight means is that both directions are machine-checked at the boundary: quarantine D_P passes and quarantine $D_P - 1$ does not. Section 14 states where that leaves the claim.

We checked it in both directions at two separated duration pairs, on the unmodified specification, with two proposers racing across acceptor crash and restart: quarantine D_P passes exhaustively and quarantine $D_P - 1$ yields a two-owner trace at each pair. Appendix D lists the configurations and their state counts.

Three consequences follow. First, waiting out the maximal lease time M after restart is safe but conservative, since it covers D_P because all durations lie below M . Second, the small bound is a dividend of the safe timer rule: under the quorum-receipt rule of Section 8 no finite quarantine suffices, so fixing the timer rule is what lets restart-and-rejoin latency track the attempt duration rather than the longest exclusion in the system. Third, quarantine is a correctness parameter and not a grace period to be tuned. One should set it to the attempt duration (adjusted for clock-rate error, as below), and if even that latency is unacceptable, then the alternatives are durable lease state or membership-level fencing, not a shorter wait.

8. Timer Placement and the Renewal Qualifier

Two of the rules of Section 4 have plausible alternatives that are unsafe, and this section gives both, each with the

two-owner execution that the model checker finds when the rule is weakened: where the proposer’s timer starts (T1), and how a proposer reads a prepare response that reports its own lease (the renewal qualifier of P2). Both are about the same question, whether evidence gathered before a crash may still be believed after it. Rule T1 starts an attempt’s deadline when the proposer sends Prepare, and the requirement behind it is more general than the rule itself: with volatile acceptor state, every piece of evidence gathered before a crash must carry a bounded lifetime from its creation, not from its later use. Four mechanisms meet the requirement and two plausible ones do not.

- **The PaxosLease mechanism, safe: rule T1.** The deadline is stored when Prepare is broadcast, and the whole attempt, its unprocessed prepare responses included, expires with it. This is the rule specified in Section 4 and checked throughout this paper. It shortens the usable lease, since network and processing delay come out of the interval.
- **Alternative 1, safe: durable promises.** Restores the invariant that an acceptor never forgets what it promised, so no evidence needs a lifetime at all. Adds a stable write on the prepare path, the very thing PaxosLease was designed to avoid.
- **Alternative 2, safe: clock-derived, globally advancing ballots.** The Fleas mechanism (Section 13). Rests on a clock-synchronization assumption, since a ballot drawn after a crash must exceed every ballot drawn before it.
- **Alternative 3, safe: a retry deadline held in protocol state** and checked in every response handler. Adds a word of state and no assumption.
- **Alternative 4, unsafe: the deadline starts at prepare-quorum receipt.** The remainder of this section; TLC ends it with two simultaneous owners.
- **Alternative 5, unsafe: a retry deadline enforced by control flow** rather than stored as state, which is what both audited implementations, Keyspace and ScalienDB (Section 11), ship. A scheduling pause deletes a deadline that exists only as dispatch order, and TLC finds the two-owner execution with the shipped constants (Section 11, third finding).

Appendix B compares them. The requirement is what transfers to other diskless protocols, and the choice among the mechanisms is local.

Alternative 4 deserves the detail, because it is where the timer lands if one takes the deadline to protect the use of the evidence rather than the evidence itself: start it when the prepare quorum is received, at the moment the evidence is about to be used. Between Prepare and quorum receipt the attempt then has no timer at all, a prepare quorum acquires unbounded shelf life, and no finite quarantine can cover what has no deadline. TLC exhibits the consequence

directly: with two acceptors and the full quarantine bound, a 27-state execution ends with two simultaneous owners, and with three acceptors and majority quorums a 25-state execution needs only a single crashed acceptor, the one in the intersection of the two proposers’ quorums. Appendix B walks the execution step by step and gives the failure in implementation terms: the exposed window is ordinary scheduling delay, so a garbage-collection pause plus one acceptor restart is the entire schedule.

Timer placement is not the only unsafe choice here. Rule P2 (Section 4), the proposer’s handler for a prepare response, counts a reported lease as an open response when the requesting proposer is the one that owns it, and the qualifier there, for a renewal only, carries safety of its own. Consider a proposer that abandons an attempt (rule P5) with its accept requests still in flight and retries later under a fresh ballot, while a competitor acquires in between under a lower one, as a first-attempt competitor against a second-attempt retriever will. The abandoned requests, landing late, overwrite the competitor’s record at every acceptor, since ballot order permits it and overwriting revokes nothing; the retrying proposer’s fresh prepare then finds “its own” lease everywhere it looks. If ownership alone makes those responses open, the proposer completes its acquisition while the competitor is still active. TLC finds the two-owner execution in 36 states at the full quarantine bound, with retry enabled (the weakened variant `StaleOwnerOpen.tla`); the executable witness replays it in the simulator; and no quarantine length prevents it, because the stale accept request can be delayed arbitrarily. Both audited implementations carry the unqualified rule (Section 11, seventh finding).

Two rules prevent this execution, one in the proposer and one in the acceptor, and each suffices alone. The proposer-side rule is P2’s renewal qualifier, stated in Section 4. The acceptor-side rule is that an acceptor never replaces a live lease of a different owner, even under a higher ballot, so the stale accept request cannot erase the competitor’s record at all; it is local and auditable, does not rely on proposers classifying their own attempts correctly, and gives up availability instead, since a live record blocks a conflicting acquisition until it expires. TLC checks both: the renewal qualifier passes the retry configuration exhaustively (Section 9), and the acceptor rule, checked with the qualifier deliberately dropped, passes the same configuration over an exhaustive 337,917,446 distinct states. Both executions of this section survive a connection-oriented transport, in which a crash discards the messages addressed to the crashed process; Appendix C gives that refinement and its verdicts.

9. Checked Models and Tests

The paper’s accompanying repository contains the TLA+ specification and its configurations, the deliberately weakened variants of it, the TLAPS proof module, two

Variant	Weakened rule	Violation	Trace
LateTimer	timer starts at prepare-quorum receipt	LeaseExclusivity	27 states
OwnerOnlyRelease	release matches owner, ignores ballot	LeaseExclusivity	36 states
ScalarQuorumCounting	quorum counted by message, redelivering transport	LeaseExclusivity	18 states
StaleOwnerOpen	own lease open without the renewal qualifier, retry enabled	LeaseExclusivity	36 states

Table 1: Counterexample variants: full copies of the specification with exactly one rule weakened, regenerated from it and compared against the checked-in files by `make variant-check`. The target `make counterexamples` requires TLC to find the first three violations on every run; the `StaleOwnerOpen` search is multi-hour and is recorded evidence, validated against the cited trace by `make paper-claims`. The `LateTimer` and `StaleOwnerOpen` configurations use the full quarantine bound.

Python models, the tests, and the recorded output of every run; [Appendix D](#) inventories it. The paper’s negative claims, that some rule cannot be dropped, are verified by modified variants of the original: copies of the specification, `tla/spec/PaxosLease.tla`, each with exactly one rule weakened and nothing else changed. A script generates the variants from the specification, so no variant can drift away from the specification it claims to weaken. TLC must then find the advertised violation in each one, also on every run (Table 1).

The positive claims are exhaustive, but only for the configurations checked, and those are small: no exhaustively checked configuration has more than two proposers, three acceptors or three ballots, and each fixes one duration triple (D_P, D_A, Q) . Inside such a configuration TLC enumerates every reachable state and finds no violation of `LeaseExclusivity`; about larger ones it says nothing, a limit whose dimensions Section 6 discusses. [Appendix D](#) lists them all, and they cover the separated-duration boundary pairs of Section 7, explicit retry at the quarantine boundary, the redelivering transport, and three feature-interaction configurations combining renewal, retry, release, and redelivery, the last added when the stale-owner execution showed that features checked in isolation are not features checked together. A further family of configurations projects the rules the two audited implementations actually carry, in particular a retry timeout in place of a stored attempt deadline, and reproduces both machine-found failures under the constants those systems ship (Section 11).

The four variants of Table 1 correspond one for one to the rules this paper says cannot be dropped. The first, `LateTimer`, is the timer placement of Section 8, whose 27-state trace [Appendix B](#) walks through in full. The second completes the release story of Section 4. If release matches on owner alone, then a release message delayed past a re-acquisition by the same owner erases the newer lease. The owner releases instance $(p_1, 1)$ and re-acquires as $(p_1, 2)$, the stale release then clears the ballot-2 lease at the acceptors, and a second proposer acquires while p_1 is still active, in a 36-state trace. The third records the deduplication stake of Section 11: with quorums counted by message under a transport that may redeliver, a single acceptor’s doubled response is a quo-

rum, and two proposers become active in 18 states with no crash, no restart, and no clock movement at all. The fourth is the stale-owner execution of Section 8: the base module with P2’s renewal qualifier dropped, whose 36-state two-owner search is the multi-hour recorded run `make counterexamples-staleowner`.

The safety argument of Section 6 rests at several points on inequalities between durations, and those inequalities are proved mechanically, by TLAPS, in `tla/proof/PaxosLeaseProof.tla`: timer containment, the quarantine bound (both forgotten facts reduce to attempt lifetime), monotonicity of the deadline cap used by the finite model, tightness of the quarantine bound, and the clock-rate condition of Section 3.

10. Runnable Demonstration

To aid understanding for practitioners, the accompanying repository contains one more program, `python/demo.py`: the algorithm of Section 4 in a single-file Python program with no dependencies, three nodes in one event loop, each hosting proposer, acceptor, and learner, on real monotonic-clock timers. Every method names the step it implements, P1 through P5, A1 through A4, L1, and the timing rules, so the file reads against the paper. Where this paper shows that a plausible alternative to a rule is unsafe, the comment at that rule says so and points at the section. The program demonstrates normal operation: a node acquires the lease, renews it at half-life, and dies; a survivor takes over once lease and quarantine have run out; and a referee object receives every learner-level ownership claim and asserts throughout that no two nodes are ever owner at once.

11. Audit of an Unformalized Protocol

The protocol of Section 4 existed as a paper and open source implementations for over a decade with no machine-checked model. This section records what went unnoticed as a result, first in the specification and then in the code. The forensic detail, trace walkthroughs and source archaeology, is in [Appendix B](#) and [Appendix C](#).

The previous paper describes the start of the proposer’s timer twice and the two descriptions disagree. Its figure

draws the safe rule, the deadline stored when Prepare is broadcast, which is the rule this paper specifies as T1; its pseudocode states the quorum-receipt rule that Section 8 shows unsafe; and its prose is consistent with both. The counterexamples are small and need nothing exotic (27 states; 25 with three acceptors and a single crashed acceptor), and [Appendix B](#) gives one in full.

PaxosLease was implemented by this author in two open source systems, Keyspace and later ScalienDB. A protocol author auditing his own fifteen-year-old code against his own checked model is an unusual exercise, so we enumerate lessons learned. Keyspace’s implementation of the algorithm lives in `src/Framework/PaxosLease/`, and ScalienDB’s in `src/Framework/Replication/PaxosLease/`. The audit reads the final commits of both repositories (a99f24a8 of 2011 and 60978146 of 2013), and produced eight findings; the details are in [Appendix C](#).

1. **Both systems implement the unsafe timer rule of the pseudocode, but an unrelated retry timeout accidentally prevents unsafe executions such as two lease holders at the same time.** Both systems implement the *unsafe* timer rule of the pseudocode, not the safe one of the figure: `StartProposing()` sets `expireTime = Now() + duration`, so the authority clock starts when the prepare quorum is in hand, exactly the rule that Section 8 shows unsafe. What both contain instead is a mechanism the previous paper never mentions, an attempt-restart timeout of $R = 2$ seconds armed when Prepare is broadcast, whose firing re-prepares under a fresh proposal identifier and thereby discards the abandoned attempt’s responses. This is the fifth alternative of Section 8, the attempt time bound enforced by control flow rather than stored as data. Neither codebase presents it as a safety mechanism; it serves as one by accident.
2. **That accidental prevention works only under certain conditions, which the implementations accidentally satisfy.** We built a checked projection of the implementations onto their lease acquisition state machine (`tla/spec/PaxosLeaseImpl.tla`) and checked when the accidental repair works. Under timely timeout dispatch the restart condition is

$$Quarantine \geq \max(D_P, R),$$

checked at its boundary in both coordinates and refuted one unit below in each; the boundary run also refutes the additive hypothesis $Q \geq R + D_P$, which an earlier draft of this section favored. The shipped ordering of the constants passes exhaustively.

3. **Specific control flow ordering of event handlers can create unsafe execution such as two lease holders at the same time.** Neither proposer stores when its attempt began; the R bound holds only because each event-loop iteration runs due timers before

socket events. The real assumption is therefore not merely that the process is never suspended, but that no message handler belonging to an expired attempt ever executes past the deadline before the timeout transition runs, and the loop structure does not enforce that: the poll blocks until the next timer is due, so a socket that becomes ready just before the deadline has its handlers, and any burst behind them, dispatched before the next timer scan. A process suspension beginning after a timer scan and ending inside the poll is the dramatic instance; ordinary event-loop overrun is the everyday one. TLC finds the resulting two-owner execution with the shipped constants in 27 states. The shipped constants are therefore not enough to call the implementations safe.

4. **Quorums are counted by message rather than by acceptor identity, which is safe only because the transport happens not to duplicate.** Both systems count quorum responses without acceptor identity, so one response delivered twice counts as two votes. We found no path that retransmits a response: pending writes are discarded on disconnect, and the inspected TCP connections supply non-duplicating delivery within a connection. The scalar counting therefore rests on an application-level at-most-once assumption that is plausible for these implementations but stated and checked nowhere, and the weakened variant `ScalarQuorumCounting.tla` shows what it protects: under a transport that may re-deliver, TLC finds a two-owner execution in 18 states.
5. **Every lease timer reads the wall clock, and neither system defends both of the directions in which a wall clock is unsafe.** Every lease timer in both systems reads the wall clock, and clock errors are not symmetric between the roles: backward or slow steps are unsafe on a proposer (authority extends) and merely prolong exclusion on an acceptor, while forward or fast steps are unsafe on an acceptor (exclusion shrinks) and harmless on a proposer. Keyspace uses `gettimeofday()` raw and is exposed in both unsafe directions; ScalienDB repairs backward steps well, through a correction thread whose behavior an earlier draft of this section misdescribed, but lets forward steps pass uncorrected. Neither system calls the monotonic clocks its platforms provided, `clock_gettime(CLOCK_MONOTONIC)` on Unix and `GetTickCount64` or `QueryPerformanceCounter` on Windows.
6. **The learner is where ownership becomes visible, and its remote expiry rests on an undocumented delivery bound.** Ownership is application-visible only through the learner, and the learner re-anchors time. In both systems the owner’s own learner installs the absolute expiry the proposer computed, so the two-owner executions above carry through to two simultaneously application-visible masters; but every

remote learner sets its expiry to arrival time plus the remaining duration minus a 500-millisecond hedge, so the hedge is conservative only while `LearnChosen` delivery takes less than 500 milliseconds, another undocumented timing assumption.

7. **A proposer treats any lease it owns as an open response even when it is no longer the active owner, which can lead to unsafe executions such as two lease holders at the same time.** Both proposers treat a reported lease they own as an open response whether or not they are currently the active owner: `StartProposing` proceeds, with a full fresh duration, whenever the discovered lease owner is the node itself. Section 8 shows the rule unsafe in combination with the retry timeout of the first finding: an accept request from an abandoned attempt can overwrite a competitor's live lease record and then serve as its sender's own justification, and TLC finds the resulting two-owner execution in 36 states at the full quarantine bound, with no dispatch delay and no clock error. The execution does not depend on connectionless delivery: under the connection-oriented refinement, in which a crash discards the messages addressed to the crashed process, TLC finds it at the same depth, with the stale accepts sent after the acceptors restart, which a reconnecting writer performs faithfully. The repair is the renewal qualifier that P2 states: an own lease is an open response only for a renewal attempt, and only for the very lease captured as its base. The composition is checked in the implementation projection itself, not only in the base model: with three ballots, the shipped timely constants, and the connection-lifecycle transport, TLC composes the shipped retry timeout with the shipped own-lease rule into a 35-state two-owner execution, found after exploring 564,425,413 distinct states (`PaxosLeaseImplStaleOwner.cfg`). Timer dispatch is timely throughout that execution: it needs no suspension, no dispatch delay, and no clock error.
8. **The activation guard reads the clock twice, and an unsigned subtraction can wrap so that a proposer activates a lease that has already expired.** The Phase 2 handlers read the clock once for the expiry guard and again for the activation-margin subtraction, on `uint64_t` deadlines. If the clock crosses the deadline between the two reads, through a pause between the calls or a forward step, the unsigned subtraction wraps, the margin check passes, and the proposer activates an expired lease; Keyspace additionally broadcasts the wrapped value as the lease duration. The window is narrow, but it is a time-of-check to time-of-use race on the safety-critical comparison, and the repair, one clock read per handler with a compare-before-subtract, is standard.

12. Lessons

Writing this paper taught the author several lessons, and none of them is specific to `PaxosLease`. They are enumerated below, in the order in which the work runs.

1. **Algorithms need to be formally specified and checked.** Lamport's standing advice¹ is to write the algorithm in TLA+ and let the model checker debug it, and this paper is a fifteen-year-late instance of taking it.
2. **The implementation should be derived from the specification.** Both audited systems were written from the pseudocode, and the pseudocode was the one place where the safety-critical rule was stated wrongly, so the diagram that had it right never reached the code.
3. **The implementation must also be re-encoded as a specification and checked.** Software contains networking and messaging code, event handling and control flow, uses specific data types to store the algorithm's variables, and various other implementation details, all of which are opportunities to introduce bugs into an otherwise correct algorithm. By translating the software implementation back to a specification and then checking it, we reduce the risk of shipping unsafe code to production.
4. **Frontier language models speed up both the derivation and the re-encoding steps.** Writing a specification, its configurations, its weakened variants and its harness has been the time expense that keeps engineers away from formal methods. Modern frontier models automate away much of this work. Extensive human guidance and review are required throughout the process.

13. Related Work

Leases as time-bounded ownership originate with Gray and Cheriton [3]. Chubby [6] made the lease-plus-Paxos architecture standard practice at scale, and the engineering account of [7] describes how much of the difficulty lies outside the core algorithm. `FaTLease` [8] solves the same lease-negotiation problem as `PaxosLease`, but it runs Paxos instances for the lease commands and assumes clock synchrony, and `PaxosLease` was designed to remove both [1]. The closest relative is `Flease` [10], from the same group as `FaTLease`: decentralized lease coordination without stable storage for the lease state, built on a round-based register.

¹In December 2009 the author described `PaxosLease` in an email to Leslie Lamport, who replied that he did not have time to look at the algorithm and suggested coding it in TLA+ and using the model checker to debug it. This paper is that suggestion, carried out seventeen years later. He was right on both counts: the model checker was the proper instrument, and the algorithm did need debugging.

Raft [4] structures leadership through terms and elections, but its safety, like that of Paxos, does not depend on real-time exclusivity. Production Raft systems that serve reads from the leader without a log round trip add a leader lease, and they inherit exactly the containment and clock-rate obligations of Section 3. Paxos Quorum Leases [12] generalize the lease holder from a single leader to a quorum of replicas that may serve local reads. They are a read-performance mechanism layered on Paxos rather than a diskless leadership protocol, but their correctness rests on the same real-time containment argument, applied at each lease-holding replica.

On the verification side, Chand, Liu, and Stoller [5] give a full TLAPS proof of Multi-Paxos, and Grove [11] is the modern benchmark for mechanized lease reasoning, verifying time-based leases together with crash recovery, reconfiguration, thread-level concurrency, and unreliable networks in an executable key-value store. Lamport’s Paxos papers [2, 9] set the specification style, and the trace-validation work of Cirstea et al. [13] shows how the remaining gap between such models and an implementation can be closed mechanically.

14. Limitations

PaxosLease assumes non-Byzantine processes that stop acting when their leases expire, and it does not prevent a faulty process from writing to an unfenced external resource.

The TLC runs are finite-state checks, no configuration with more than three acceptors was checked, and each configuration also caps the number of messages in flight. They do not prove the protocol for all quorum systems or all cluster sizes. Non-majority quorum systems are unchecked altogether.

The TLAPS file proves the arithmetic lemmas of the timing argument, not the theorem that the transition system maintains exclusivity. That theorem rests on three legs, none of them a proof of it: the argument of Section 6, the lemmas, and finite model checking. Section 7 shows what the combination can miss, since a bound that was too conservative survived an exhaustive check because the configurations could not separate the right answer from the wrong one.

The implementation audit of Section 11 is a reading of the sources against the checklist of the model, backed by a checked projection of the implemented protocol and an executable abstraction of its node. We did not construct the described executions against running binaries, and the quantitative claims about margins assume the shipped compile-time constants.

The clock model assumes monotonic elapsed-time measurement with bounded rate error. A process that is suspended, paused by its hypervisor, or live-migrated stops running while real time keeps passing, so its lease can expire before it next looks at its timer, and it may go on acting

as owner when it resumes. This is a separate concern from the failure of Section 8, which needs no clock misbehavior at all.

Retry chains are bounded by the three ballots of the checked configurations; a four-ballot configuration, admitting two consecutive retries beside a competitor, was terminated at a time cap after roughly 2.3 billion distinct states without a verdict.

The acceptor set is fixed. Reconfiguration requires preserving quorum intersection across configurations, or else waiting until leases from the old configuration can no longer matter.

15. Conclusion

PaxosLease grants time-bounded exclusive ownership from a quorum of acceptors that keep no durable lease state. Its standard use is to elect a Multi-Paxos leader, which runs Phase 1 once per epoch and then appends with one round trip and one durable write per value for as long as it renews the lease. Safety rests on quorum certificates whose acceptor exclusions outlast the proposer’s authority, on ballots that order competing attempts within an epoch, on a quarantine of one attempt duration that makes forgotten state safe across epochs, and on release and renewal rules that keep a proposer’s own stale messages from impersonating its authority. Each clause is paired here with a machine-found execution showing what happens without it.

The work also suggests a way of building such systems: specify the algorithm formally and check it before trusting it; derive the implementation from that specification; encode the implementation, with its transport, its event loop and its data types, as a specification and check it in turn. Frontier language models make both encodings fast enough to be routine, with human direction and review throughout.

Lamport’s advice, given to the author in 2009 and taken seventeen years late, was right on both counts: the model checker was the proper instrument, and the algorithm did need debugging.

Large language models assisted in constructing and revising these models and programs, under the direction and review of the author.

Appendix A. Checked Invariants

In this appendix the invariant and obligation names are written with spaces, `Lease Exclusivity` rather than `LeaseExclusivity`, to improve readability. In the TLA+ sources and in the body of the paper they are the corresponding single-word identifiers.

Appendix A.1. Standalone PaxosLease Invariants

Type OK: Every variable remains in its declared finite domain: times are bounded, messages have one of the modeled shapes, acceptor state maps acceptors to lease records, proposer phases are among the modeled phases, and response sets are sets of acceptor identities. This is the well-formedness condition needed before the other predicates have their intended meaning.

Accepted Coherence: An acceptor’s accepted value is either `NoLease` or a lease whose owner, ballot, and deadline lie in the modeled domains. This rules out malformed accepted state and ensures that later prepare responses can be interpreted as lease reports.

Active Implies Unexpired: Every active proposer has a local deadline strictly greater than the current model time. This captures the rule that a proposer stops acting when its own lease interval expires.

Lease Exclusivity: The set of active proposers has cardinality at most one. This is the main safety property. It is checked directly by TLC, checked by the Python simulator after every operation, and violated on purpose by every counterexample variant.

Activation Has Quorum: An active proposer has a quorum certificate. A proposer cannot become active merely because it sent requests or received a non-quorum set of responses.

Timer Containment: For every acceptor in an active proposer’s certificate, the proposer’s deadline comes no later than the acceptor deadline recorded in the certificate. The certificate variables are history variables (Section 3), so the invariant is a statement checked by the model, not a comparison performed by the protocol.

Quarantine Prevents Participation: An acceptor in quarantine has no promised ballot and no accepted lease. Together with the transition rules, this means that a restarted acceptor cannot vote while forgotten lease state could still matter.

Active Ballot Well-Formed: An active proposer’s active ballot is one of the modeled ballots. This is a bookkeeping invariant for the certificate machinery. The substantive renewal property, that a failed renewal does not extend authority, is a transition rule exercised by the renew/release configuration and by the scenario tests.

Appendix A.2. PaxosLease+Paxos Invariants

Chosen Value Well-Formed: The abstract composition model represents each slot by a single chosen value or by `NoValue`, and the transition relation assigns a chosen value only when the slot is empty. Agreement is therefore enforced by construction in this abstraction, the named invariant checks domain membership, and the substantive checks are the admission invariants below. We state this explicitly to avoid overclaiming, since the composition model checks the lease/Paxos boundary, not Paxos itself.

Ready Implies Active: A ready leader holds a live lease. This is the admission side of the composition, since a process without lease authority cannot be ready to accept client log work.

Ready Leader Uniqueness: At most one proposer is in the ready state. This is stronger than Paxos requires for agreement, but it is the real-time leadership property that the lease layer supplies.

Recovery Precedes Admission: A proposer cannot apply or admit log work before it has completed recovery. This is the modeled boundary between the lease protocol and Paxos recovery.

Ready Implies Recovered: A ready leader has completed recovery within its current lease epoch, since acquiring a lease clears the recovery mark. This fences the fast path per epoch: no leader appends on the strength of a previous epoch’s recovery.

Applied Implies Chosen: Every slot applied by any proposer has a chosen value, so the leader-admission abstraction cannot invent applied log

entries that Paxos has not chosen. **Log Prefix Consistency** is the two-proposer restatement of the same fact, implied by it and retained as a separate named check for readability.

Appendix A.3. TLAPS Obligations

Timer Containment Arithmetic: If the proposer starts its timer no later than each certificate acceptor starts its exclusion, and its duration is no longer, then the proposer’s authority interval is contained in the acceptor’s exclusion interval.

Quarantine Covers Forgotten Facts: Everything an acceptor can forget, whether a promise or an accepted lease, belongs to a proposer attempt whose `Prepare` was sent no later than the crash, and under the prepare-time timer rule that attempt is unusable D_P later. Quarantine at least D_P therefore outlasts every attempt that any forgotten fact could still support. The obligation has no analogue under the step-3 timer rule.

Impl Quarantine Covers Forgotten Facts: The analogue for the implemented protocol under timely timeout dispatch: a forgotten promise anchors at a `Prepare` and is unusable R later, a forgotten accepted lease anchors at a prepare-quorum receipt and is unusable D_P later, and both anchors precede the crash, so quarantine at least $\max(D_P, R)$ covers both cases.

Cap Monotone: The deadline cap that keeps the finite model bounded is monotone, so capping preserves containment.

Quarantine Bound Tight: Any quarantine strictly below the proposer duration leaves an elapsed interval inside a forgotten attempt’s lifetime but outside quarantine. TLC turns this arithmetic gap into the concrete two-owner traces of Section 7.

Drift Containment Arithmetic: If the proposer’s clock runs at rate at least r_{min} , the acceptor’s at most r_{max} , and the durations satisfy $D_P \cdot r_{max} \leq D_A \cdot r_{min}$, then real-time containment holds. This is the cross-multiplied form of the inequality of Section 3, and the same arithmetic with the quarantine in place of D_A gives the real-time quarantine bound.

Appendix B. Inconsistency in the Original Statement

The quarantine bound of Section 7 has a violating execution of its own, and it is short enough to give in full. To see why the bound is necessary, consider the violating execution that TLC finds. Proposer p_2 completes an acquisition and is active. Both of its certificate acceptors crash at $t = 0$, forgetting the lease and their promised ballot 2; they restart, and their short quarantine ends at $t = 1$, while p_1 ’s attempt, whose `Prepare` was sent at $t = 0$ and which therefore lives until $t = 2$, is still fresh. p_1 ’s accept request, still in flight, now lands on acceptors that remember neither the lease nor the higher promise, and it is accepted although it carries the lower ballot. At $t = 1$ both proposers are active. Observe that ballot ordering offers no protection here, since monotone promises are exactly the state the crash erased. The time-based quarantine is the only line of defense, but it needs to last only as long as an attempt.

This appendix carries the forensic detail behind Section 8 and the first lesson of Section 11.

The original paper describes the start of the proposer’s timer twice, and the two descriptions disagree. The timing diagram (Figure 2 of [1]) places “start timer” on the proposer’s lane before the prepare requests leave. The pseudocode (step 3, `Proposer : OnPrepareResponse`) starts the timer only when a quorum of empty prepare responses has been received, immediately before the propose requests are sent. The prose of the proof section says only that “the proposer starts its timer before sending out propose requests,” which is consistent with both. For thirteen years the difference seems to have attracted no attention, and this is understandable, since in every execution in which acceptors keep their state both rules are safe, and the pseudocode rule wastes less of the lease.

The difference is not benign. Under the pseudocode rule there is an interval, after `Prepare` is sent and before the prepare quorum is processed, during which the attempt has no timer at all, so a prepare quorum has unbounded shelf life. The restart rule of the original paper, that a crashed node waits out the maximal lease time M before rejoining, bounds the life of forgotten exclusions and of attempts whose timers have started. It cannot bound an attempt that has no timer, and no finite quarantine can.

Model checking makes this concrete. `tla/counterexamples/LateTimer.tla` is a copy of the specification with a single change: the pending deadline is set in the accept-sending action rather than the prepare-sending action, which is exactly the pseudocode’s rule. Under a configuration with two proposers, two acceptors, and the full quarantine bound, TLC violates `LeaseExclusivity` with a 27-state trace. In protocol terms, with $D_P = D_A = \text{Quarantine} = 2$, the execution is the following.

1. At $t = 0$, proposers p_1 and p_2 send `Prepare` with ballots 1 and 2. Acceptors a_1 and a_2 promise both, in ballot order, and both proposers receive empty promise quorums. Neither proposer has started a timer.
2. a_1 and a_2 crash, losing their promised ballots. They restart and stay silent through the entire quarantine, until $t = 2$.
3. At $t = 1$, both proposers process their prepare quorums. Following step 3 of [1], each now starts its timer, with deadline $t = 3$, and sends `Accept`.
4. At $t = 2$, with quarantine over, the acceptors process the accept requests. Their promised ballots were erased by the crash, so they accept ballot 1 and then overwrite it with ballot 2.
5. Both proposers receive accepted quorums and activate. At $t = 2$, p_1 and p_2 are both active with deadline $t = 3$.

Every step is legal for the pseudocode. No message was lost, no clock misbehaved, and every acceptor served its full quarantine. The proof of lease invariance in [1] does not cover this execution because it never considers acceptors forgetting state. Forgetting is delegated to the restart wait, and the restart wait silently assumes that every in-flight attempt is bounded by a timer, which is precisely what step 3 fails to guarantee.

It is worth restating the failure in implementation terms, because it requires nothing exotic from the network. The window between receiving the last prepare response and sending the propose requests is ordinary scheduling delay. A garbage-collection pause, a preempted virtual machine, or a page fault inside the election handler, together with the crash and restart of two acceptors, is the entire failure schedule. Implementations tend to be careful about clock rates and careless about this window, precisely because within one process it looks like a no-op.

Two repairs work. The repair the algorithm adopts as rule T1 is the rule that Figure 2 of [1] draws: start the pending deadline when `Prepare` is sent. The whole attempt, including its unprocessed prepare responses, then expires with the deadline, and a quarantine of D_P outlasts every attempt that a forgotten promise could still complete (Section 7). Network and processing delays then come out of the lease interval, so a proposer in a slow environment holds a correspondingly shorter usable lease. The other repair is to make promises durable, which restores the invariant that an acceptor never forgets what it promised, and requires a synchronous disk write on the prepare path, the very thing PaxosLease was designed to avoid. (An expiry token attached to prepare responses is the first repair in different clothing, and Fleas’s clock-derived ballots are a third strategy resting on a stronger clock assumption; see Section 13.) A fourth is a retry deadline held in protocol state and checked in every response handler, which adds a word of state and no assumption. The audited implementations approximate that fourth one with a retry timeout whose deadline is control flow rather than stored state, which Section 11 shows to be insufficient.

Section 11 and Appendix C audit the Keyspace and ScaliendB implementations against this distinction. The deterministic Python model reproduces the trace as the `late-promise-reuse` witness, where a `Simulator` flag selects the step-3 rule, and a paired test checks that the prepare-time rule blocks the identical schedule: the stale promise quorums arrive after both pending deadlines have expired, no `Accept` is ever sent, and nobody activates.

The LateTimer violation is not an accident of the two-acceptor quorum system. With three acceptors and majority quorums of two, TLC finds a 25-state two-owner trace, shorter than the two-acceptor one and needing less failure: a single acceptor, sitting in the intersection of the two proposers’ quorums, crashes, forgets its promise, serves the full quarantine, and then participates in both accept quorums. The search explored 151,146,929 distinct states under symmetry reduction on proposers and acceptors. The quarantine bound behaves the same way, since with three

acceptors and quarantine one unit below D_P , TLC finds a 24-state two-owner trace. The three-acceptor configurations are recorded in the repository and excluded from the default regression targets because of their run time; the fail-fast target validates their recorded traces and state counts against the numbers cited here, and `make paper-evidence-full` reruns them.

Appendix C. Audit Detail: Keyspace and ScaliendB

Whether these executions survive real transports is a fair question, since a connection-oriented transport discards undelivered inbound data when its receiver dies, and the audited writers additionally discard pending output on disconnect (Section 11, fourth finding). The models answer it with a refinement rather than an argument. With `CrashDropsIncoming`, messages addressed to a process are discarded both when it crashes and when it restarts, so every message that is delivered was provably sent after its addressee’s most recent restart, which is what delivery over a freshly established connection means; messages already sent by a crashed process survive, since bytes on the wire outlive their sender. The verdicts do not change. The late-timer execution survives unmodified at 27 states, because its stale messages are promises, sent by the acceptors before they crash. The stale-owner execution survives at the same 36 states, and the recorded trace shows the mechanism plainly: the acceptors crash and restart first, and the abandoned attempt’s accept requests are sent afterward, on fresh connections, which a reconnecting writer performs faithfully. The implementation projection agrees at both of its scales (Section 11, third and seventh findings, Appendix C). What the refinement establishes is where the transport genuinely carries safety weight: the at-most-once delivery of the fourth finding, not connection teardown.

This appendix carries the source-level evidence for the findings of Section 11. All quotations refer to the commits and checksums recorded by the repository’s fetch script.

Appendix C.1. The timer rule and the retry timeout

In both systems `StartProposing()` sets `expireTime = Now() + duration` (`Keyspace PLeaseProposer.cpp`, `ScaliendB PaxosLeaseProposer.cpp`), so the authority clock starts when the prepare quorum is in hand; the figure’s prepare-time rule appears in neither codebase. The attempt-restart timeout is `ACQUIRELEASE_TIMEOUT`, 2 seconds in both systems, armed by `StartPreparing()`; when it fires, `OnAcquireLeaseTimeout()` calls `StartPreparing()` again, which draws a fresh proposal identifier, and the identifier comparison at the top of every response handler discards the messages of the abandoned attempt. The maximal lease time is `MAX_LEASE_TIME`, 7 seconds in Keyspace and 3 in ScaliendB; both the lease duration and the startup quarantine are set to it. Both constants live as compile-time `#defines` in the same header as the retry timeout.

The Phase 2 handlers carry the unsigned-arithmetic race of the eighth finding. `OnProposeResponse` first tests `state.expireTime < Now()` and returns if the lease has expired, then, in a separate expression with a separate clock read, tests `state.expireTime - Now() > 500` before activating; `expireTime` is `uint64_t` (`PLeaseState.h`). A clock that crosses the deadline between the two reads makes the subtraction wrap to nearly 2^{64} , which passes the margin test, and Keyspace’s activation path then passes `state.expireTime - Now()`, a third read, to `LearnChosen` as the lease duration. The `unsigned-expiry-underflow` witness records the arithmetic.

Appendix C.2. The implementation model and its verdicts

The module `tla/spec/PaxosLeaseImpl.tla` differs from the base specification in exactly the two audited rules: `pendingDeadline` is set in `SendAccept` rather than `StartAcquire`, and an attempt deadline of R units, set at `StartAcquire`, abandons the ballot when it fires. The constant `TimelyDispatch` selects the timeout semantics: under timely dispatch, advancing time abandons every overdue attempt before anything else happens, which models an event loop that is never paused, while under delayed dispatch expiry merely enables an abandonment action that may be postponed past any amount of message processing, which is what

a suspension produces. The Phase 2 delivery action deliberately carries no attempt-deadline guard, matching `OnProposeResponse`, which checks only the lease expiry.

At $(D_P, R, Q) = (2, 2, 2)$ the timely model passes all safety invariants over an exhaustive 37,160,904 distinct states, and at $(1, 2, 2)$, where the retry timeout dominates the attempt duration, over 37,476,080. One unit below, $(2, 2, 1)$ and $(1, 2, 1)$ each yield a two-owner trace in 26 states. Since the boundary passes, the condition the shipped constants must satisfy is $R \leq M$, not $R < M$; both implementations satisfy it strictly. With the shipped ordering scaled to $(D_P, R, Q) = (2, 1, 2)$, the timely model passes over 18,160,464 distinct states, and the same constants under delayed dispatch yield a two-owner trace in 27 states. The coverage lemma behind the bound, that everything a crashed acceptor forgot anchors either at a `Prepare` (unusable R later) or at a prepare-quorum receipt (unusable D_P later), each no later than the crash, is the TLAPS obligation `ImplQuarantineCoversForgottenFacts`.

One objection to the two-acceptor delayed-dispatch trace deserves its own configuration. In the deployed systems, proposer and acceptor roles are colocated in one process, and a node crash restarts both while bumping the durable restart counter spliced into the node's proposal identifiers, so a schedule in which a proposer-hosting node crashes cannot rely on that proposer keeping its old ballot ordering. The two-acceptor trace crashes both acceptors, and in a real two-node deployment those crashes would restart both proposers. The configuration `PaxosLeaseImplDelayedColocated.cfg` therefore uses three acceptors and restricts the crashable set (a constant of the module) to the single acceptor in the intersection of the two proposers' quorums: in any trace it finds, no proposer-hosting node ever crashes, no restart counter moves, and the node-identity tiebreak between equal attempt counters is legitimate, so the trace is valid for the colocated deployment without modeling restart counters. TLC finds the two-owner execution in 25 states, after exploring 262,728,880 distinct states; it is the recorded trace behind the schedule of the next subsection. The colocation-valid execution also survives the connection-lifecycle transport refinement of Section 8: with messages to the crashed acceptor discarded at its crash and at its restart, TLC finds a 25-state two-owner execution after exploring 284,288,181 distinct states, so connection teardown does not mask this finding either.

Appendix C.3. The violating schedule, exactly

The dispatch assumption is weaker than it looks even without a suspension. The loop body is `RunTimers()` followed by `IOProcessor::Poll()` with a sleep bounded by the next timer deadline, so a timer coming due while the poll blocks is dispatched only at the next iteration's scan; a socket that becomes ready just before the deadline has its handlers, and any burst of ready events behind them, executed first. Timely dispatch would require that no handler belonging to an expired attempt run past the deadline before the timeout transition, and nothing enforces that. A suspension is simply the largest available overrun, and we use it for the concrete schedule because it makes the window arbitrarily wide.

The execution is tightly constrained, and the obvious version of it fails: if the process merely resumes after a long pause, the overdue retry fires at the next iteration's timer scan, rotates the proposal identifier, and the stale responses are discarded. The execution that works pauses *inside* an iteration, and colocation constrains it further, since a node crash would restart the colocated proposer and bump the restart counter in its identifiers. The schedule therefore crashes exactly one node, the acceptor in the intersection of the two quorums, which hosts neither active proposer. Node 2's proposer completes its prepare round against acceptors 1 and 2, starts its authority interval, and sends its accept requests, which are delayed; its loop performs the last timer scan before the retry deadline and blocks in the poll; the suspension begins there and outlasts node 1's quarantine. Node 1 alone crashes after promising, restarts, and serves out its full quarantine; node 0's proposer then completes an entire fresh acquisition against acceptors 0 and 1, and its learner installs the absolute expiry: node 0 is the application-visible master. Node 2's stale ballot exceeds node 0's fresh one because the attempt counters are equal, neither node restarted, and the tie breaks on node identity, exactly the counter-major layout. The stale accept requests are then accepted at node 1 (overwriting the record of the fresh lease without revoking its holder's authority) and, on resume, at node 2's own acceptor; the queued

responses are dispatched before the overdue timeout callback, the handler finds the lease expiry unreached with more than its 500-millisecond margin remaining, the proposer activates, and its own learner installs the absolute expiry. Nodes 0 and 2 are now simultaneously application-visible masters. A recorded run reproduces this execution step by step against an executable abstraction of the shipped node, proposer, acceptor, and learner colocated (`python/paxoslease/event_loop.py`, the `suspended-event-loop` witness, asserted at `IsLeaseOwner()` level), with a paired run showing that the identical schedule without the suspension is refused; the colocation-valid TLC configuration of the previous subsection finds the same class of execution by search. Exceeding ScalienDB's one-second margin requires only a virtual-machine pause, a swap stall, a `SIGSTOP`, or a sufficiently long handler burst.

Appendix C.4. Quorum counting and the transport

Keyspace's proposer counts scalars (`numReceived++`, `numAccepted++`), and ScalienDB's vote object takes a node identifier but uses it only for a membership test before incrementing a scalar (`MajorityQuorum.cpp`), so in both systems one acceptor's response delivered twice counts as two votes. Whether this is a defect turns on the transport. We found no path that retransmits a response: pending writes are discarded on disconnect (`TCPCConn::Close` in Keyspace, `TCPConnection::Close` in ScalienDB), and TCP supplies non-duplicating delivery within a connection. A complete proof of logical-message at-most-once across connection replacement would additionally have to exclude simultaneous old and new connections for one peer, duplicate request generation, reconnect paths that regenerate a response, and handler re-entry, which we have not done exhaustively; the counting therefore rests on an application-level at-most-once assumption that is plausible for these implementations but stated and checked nowhere. Substituting any transport that can re-deliver, and Keyspace's tree contains an unused UDP transport, would break quorum counting silently. The implementation contract now states the rule disjunctively: count by identity, or document and preserve at-most-once delivery per response.

Appendix C.5. The learner path

Application code never reads the proposer's state; it asks the local learner (`IsLeaseOwner()` in `PLearner.cpp` and `PaxosLeaseLearner.cpp`), which is populated by the `LearnChosen` broadcast that activation sends to every node, the sender included. Both learners implement the same asymmetric rule: if the learned owner is this node, the learner installs `msg.localExpireTime`, the absolute expiry the proposer computed, so the owner's application-visible authority preserves the proposer's deadline exactly, and the two-owner executions of this appendix are two-master executions at the application level once each owner's local `LearnChosen` is dispatched. For every other node the learner installs `Now() + msg.duration - 500`, re-anchoring the remaining duration at arrival time; the source comments call the 500-millisecond subtraction a conservative estimate, and it is, but only while delivery takes less than 500 milliseconds. A `LearnChosen` delayed by $d > 500$ ms leaves every remote learner believing in the owner's authority $d - 500$ ms past its real end. Remote learners do not act as owner, so exclusivity is not directly at stake, but master lookups and election suppression read exactly this state, and the delivery bound they depend on is documented nowhere.

Appendix C.6. Clock call paths

In Keyspace, `Now()` is `GetMilliTimestamp()`, which is `gettimeofday()` raw (`Platform.cpp`), with no clamp and no correction. In ScalienDB, every lease-layer call reaches the `Now()` of `Time.cpp`: `gettimeofday()` plus a global correction offset, with a per-thread monotonic clamp, and a clock thread, started unconditionally at boot (`Main.cpp`), samples the clock every 5 milliseconds and, on detecting a backward step, permanently raises the correction offset so that corrected time resumes at its previous value plus one resolution step. An earlier draft claimed that ScalienDB's clamp turns a backward step into a clock that advances slowly for the length of the step; tracing the call path shows that is wrong for the deployed binary, since the correction thread

repairs the step within one 5-millisecond period and elapsed intervals measured across it remain approximately correct. Forward steps pass through uncorrected in both systems. (The `raw_gettimeofday()` failure is the `unsafe-clock-source` witness of the repository, verbatim.) ScalienDB comes closest to a monotonic clock by accident: its Windows path in `Time.cpp` builds an elapsed-time scheme on `timeGetTime()`, milliseconds since boot, with a correction that keeps it increasing, and then never runs it, since the enclosing `_gettimeofday()` opens with `return_gettimeofday_win(tv, NULL);` and everything below that return is unreachable.

Appendix C.7. What the implementations get right

The restart quarantine exists in both, at the full *M*: Keyspace boots with its lease transport reader stopped for `MAX_LEASE_TIME`, and ScalienDB drops every lease message while its startup timeout is active. The durable restart counter is exactly the footnote of [1]: Keyspace persists `@restartCounter` in its table store, ScalienDB increments and commits `runID` at boot, and both splice the counter into the proposal identifier below the attempt counter, which is the same counter-major layout as this paper’s Python ballots. ScalienDB additionally raises its proposal counter above any identifier observed in incoming requests, the catch-up rule that Section 3 recommends. Acceptors in both systems lazily expire accepted state before answering a prepare, matching the report rule of the model. Finally, neither system implements release at all; release messages exist only in the paper.

Appendix D. Repository Inventory

This appendix carries the inventory behind Section 9: the specification and its options, every configuration with its state count, the mapping of each rule to the models and tests that check it, the implementation-projection and composition models, and the Python testing layers.

The main TLA+ specification is `tla/spec/PaxosLease.tla`. It contains actions for prepare, promise, accept, accepted, release, attempt abandonment and retry, the advance of time, proposer crash and restart, acceptor crash and restart, and, when enabled, message loss. Messages are represented as a set, so identical concurrent sends collapse, and delivery normally consumes the message: an at-most-once transport by construction. The `AllowRedeliver` option removes that assumption, letting every delivery nondeterministically leave the message in the network so that the same request or response can be delivered again arbitrarily later. Two configurations check safety under that transport: a crash-free one that isolates the counting question, and a recorded exhaustive run that re-adds acceptor crash and restart at the quarantine boundary. Both hold because quorum responses are counted by distinct acceptor identity; the scalar-counting variant below runs the same transport with responses counted by message and loses exclusivity in 18 states. TLC’s ballots are an ordered scalar set with a global used-ballot history; the counter-major (*counter, restart, node*) layout of Section 4 is the concrete refinement used by the Python models and by the implementations, and the model checks only what any layout must provide, uniqueness and total order. The timing assumption under which the protocol is safe, quarantine at least the proposer duration, is stated in a thin wrapper module, `PaxosLeaseChecked.tla`. Safe configurations check the wrapper, while the intentionally unsafe configuration runs the raw module, so that TLC exhibits the resulting violation instead of rejecting the configuration.

Every TLC configuration bounds the number of in-flight messages (`MaxNetwork`). This is a real caveat rather than a footnote, since a state constraint removes executions, and “exhaustive” below always means exhaustive for the constrained state graph. Two mitigations apply. For the violation runs the caveat is irrelevant, because a violating trace is a violating trace. For the passing runs we measured the sensitivity to the bound. The two-acceptor crash and boundary configurations have identical state counts at bounds 4 and 5, so the bound is never reached and those checks are exhaustive without qualification, while the three-acceptor Base configuration grows from 573,975 through roughly 3.5 million to 8.6 million states at bounds 4 through 6 with no change in verdict. For the headline retry configuration the bound is checked rather than trusted: the identical configuration at `MaxNetwork = 5` is exhaustive at 465,991,204 distinct states with the same passing verdict, and the violating executions of this

paper need at most four concurrent messages, an abandoned attempt’s accept pair beside a fresh attempt’s prepare pair.

The stale-owner result is also a lesson about configurations: it appeared only when retry was modeled explicitly, so features checked in isolation are not features checked together. Three interaction configurations therefore combine what the isolated ones separate, and all three pass: renewal with retry and crash (280,165,306 distinct states, exhaustive), retry with the redelivering transport (53,723,103), and renewal with release, retry, and room for stale Phase 2 traffic (204,050).

Not every rule of Sections 4 and 5 lives in the base module, and the evidence architecture is honest about which model or test carries which rule. Table D.3 states the mapping. The proposer and acceptor transitions and the timing rules are base-module territory, checked in every configuration. The learner rule L1 and the clock discipline are implementation-layer rules: they concern how activation and expiry are surfaced to an application, which the base module abstracts away, so their evidence is executable witnesses and the source audit. The leased-leader lifecycle is carried by the composition model and the executable Python model together.

The implementation-projection module `tla/spec/PaxosLeaseImpl.tla` of Section 11 is checked the same way, in seven configurations: three passing boundary configurations under timely timeout dispatch, including the shipped ordering of the constants, and four violating ones, quarantine one unit below the attempt duration, retry timeout exceeding the quarantine, and the shipped constants under delayed dispatch in both the two-acceptor form and the colocation-valid three-acceptor form in which only the quorum-intersection acceptor may crash (`make check-impl`; the multi-hour colocation-valid search, like the three-acceptor counterexamples, is recorded evidence that the fail-fast `make paper-evidence` target does not re-run. Instead, every evidence run cross-checks the recorded traces and state counts against the numbers this paper cites, via `python/scripts/check_paper_claims.py`, and `make paper-evidence-full` re-runs the long searches themselves). Its module header documents, rule by rule, which audited source line each deviation from the base specification models.

The two models check the lifecycle from opposite ends. The TLA+ composition is the abstract barrier: a ready leader must hold a live lease, at most one leader is ready, admission requires readiness, and readiness requires this epoch’s recovery, so fast-mode appends are fenced exactly as the readiness and fast-mode steps M4 and M5 of Section 5 state. The Python model is the concrete half: the recovery, repair, and append steps M2, M3, and M5 run real prepare, repair, and accept rounds against ballot-checking durable acceptors, and the tests assert that admission is faithful. An earlier version of that model let the first client append complete the repair, so that `append(client)` could return a recovered old value in place of the client’s command, which is exactly the mistake M3 forbids; the current tests reject it. The Paxos side of the composition checks that applied slots correspond to chosen slots and that the chosen prefix is consistent. This is a non-interference check, not a proof of Multi-Paxos linearizability: the abstraction contains no ballots, so the theorem that skipping `Prepare` preserves slot agreement is Paxos’s own [2, 9] and is not re-proved here (slot agreement is enforced by construction, and the invariant named `ChosenValueWellFormed` is a domain check). A full proof would also model client invocations and responses, duplicate suppression, reads, holes in the log, reconfiguration, and refinement to a sequential specification. We present the composition model as an explanatory contract: it checks the boundary between the layers, and the deep verification burden on the Paxos side is out of scope.

The Python layer is the repository’s test bench: an executable reference model, not a deployable implementation (the runnable file of Section 10 serves first contact instead). It encodes the same rules a second time, separately from the TLA+ though by the same hands, so that a wrong rule has to survive two unrelated encodings, and it asserts the safety invariants after every step of every schedule it runs. Where TLC searches all schedules within bounds, the Python runs specific ones: the deterministic simulators replay the adversarial stories, and the witness suite keeps every known failure executable. The layer contains two deterministic models with virtual time. The lease-only model exposes explicit operations for starting an acquisition, delivering, dropping, or duplicating a message, advancing time, crashing or restarting an acceptor, crashing or

Configuration	(D_P, D_A, Q)	Purpose	Distinct states	Result
Base	(2, 2, 2)	acquisition races, two proposers, three acceptors	573,975	pass
Renew/release	(2, 2, 2)	renewal and exact-instance release	7,717	pass
Crash/restart	(2, 2, 2)	two proposers race across crash+restart	10,059,404	pass
Drift	(1, 2, 2)	drift-adjusted (shorter) proposer duration	19,656	pass
Quarantine = D_P	(1, 2, 1)	boundary: quarantine below acceptor exclusion	20,447,948	pass
Quarantine = D_P	(2, 3, 2)	boundary at a second duration pair	9,867,548	pass
Redeliver	(2, 2, 2)	redelivering transport, identity counting, no crash	1,857,563	pass
Retry	(2, 2, 2)	explicit abandon and retry, three ballots, crash	251,904,392	pass
Renew+Retry	(2, 2, 2)	renewal and abandon together, crash	280,165,306	pass
Retry+Redeliver	(2, 2, 2)	abandoned attempts under redelivery	53,723,103	pass
Renew+Release+Retry	(2, 2, 2)	stale Phase 2 traffic around renewal and release	204,050	pass
Lease+Paxos	n/a	abstract composition barrier	2,095	pass
Unsafe quarantine	(1, 2, 0)	quarantine one unit below D_P	not exhaustive	violated (25-state trace)
Unsafe quarantine	(2, 3, 1)	quarantine one unit below D_P, D_A larger	not exhaustive	violated (26-state trace)

Table D.2: TLC checks of the unmodified specification. Passing counts are exhaustive for the finite configurations, subject to the per-configuration bound on in-flight messages. Violated runs stop at the counterexample; their trace lengths are minimum-depth under breadth-first search and stable across runs, while the number of states a parallel search happens to explore before finding the violation is not, so it is not cited. The two boundary rows are the separated-duration experiments that establish $Quarantine \geq D_P$ as the tight bound in the checked model. The Retry and Renew+Retry rows, at a quarter-billion states each, are multi-hour recorded runs; the rest run in the fail-fast pipeline.

Rule	Base module	Variant/projection	Python witness	Source audit
Attempt time bound (P1, T1)	StartAcquire stores d	LateTimer	simulator, timer witnesses	unsafe rule shipped
Retry time bound (P5)	AbandonAttempt, Retry cfg	impl projection (R)	n/a	undocumented $R \leq M$
Exact-instance release (A3)	DeliverRelease	OwnerOnlyRelease	release witness	no release path shipped
Restart quarantine (A4, T3)	RestartAcceptor	unsafe-quarantine cfgs	quarantine witness	full- M quarantine
Identity counting (P2)	response sets, Redeliver	ScalarQuorumCounting	duplicate witness	scalar counters, TCP accident
Own-lease-open qualifier (P2)	DeliverPromise	StaleOwnerOpen	stale-owner-open witness	unqualified rule shipped
Learner deadlines (L1)	not modeled	n/a	event-loop witness	remote re-anchoring
Clock discipline	not modeled	n/a	underflow witness	double clock read
Leader lifecycle (M1–M5)	composition model	n/a	leased_paxos.py, tests	recovery conflation

Table D.3: Each rule of Sections 4 and 5 mapped to the models and tests that check it. “Not modeled” marks the implementation-layer rules that the base transition system deliberately abstracts away; their evidence is executable witnesses and the audit of [Appendix C](#).

restarting a proposer, and releasing a lease, and a constructor flag selects the unsafe quorum-receipt timer rule for the `late-promise-reuse` witness (Appendix B). The composed model contains separate volatile lease acceptors and durable Paxos acceptors, and a process may append a client value only when it holds a live lease and has completed Paxos recovery. The correspondence between the TLA+ actions and the Python operations is documented file by file. Where the two models once disagreed (an early version of the specification let a quarantined acceptor process release messages, which the Python model refused), the specification was corrected to match the stated semantics.

The Python tests include scenario tests, Hypothesis-generated schedules, and a Monte Carlo style schedule search. The Monte Carlo driver chooses randomly among protocol operations, with delivery weighted more heavily than fault injection (under uniform choice almost no schedule completes an acquisition, so the interesting states were barely visited), and half the schedules begin with a completed acquisition so that crashes, restarts, and stale messages exercise a live lease. After every operation the driver checks the same implementation invariants used by the deterministic simulator, and it reports meaningful transitions rather than raw scheduler picks: effective state-changing steps, completed activations, acceptor crashes while a lease was active, and restarts with messages still in flight. Randomized search is supporting evidence here, not a headline result.

The repository also records fifteen structured negative witnesses, each labeled with the kind of evidence it is: deterministic executions (insufficient quarantine; late promise reuse, the executable twin of the `LateTimer` trace; and the suspended event loop of Section 11, with its paired timely run), a composed-model execution (skipped Paxos recovery), illustrative executable examples (owner-only release, unfenced external effects, non-intersecting reconfiguration), and unit assertions connecting a local rule to its failure (duplicate quorum counting, ballot reuse after restart, unsafe clock sources, renewal without quorum, stale lower-ballot accept, split-brain reads, unsigned expiry underflow). Each witness must fail with its specific expected message, so an unrelated failure cannot masquerade as the expected one. The witnesses connect implementation rules to concrete failures, while the TLC counterexamples of Table 1 are the stronger class, in which the model checker finds the violating schedule itself.

Finally, a smoke test records one happy-path acquisition through the trace recorder of the Python simulator and checks event coverage and invariants. We previously called this trace validation, and it is not, since it does not relate implementation traces to the TLA+ transition relation. Genuine trace validation, in the sense of [13], reduces recorded implementation traces to a constrained model-checking problem, and applying that technique here is future work.

Appendix E. How This Paper Was Written

The models, the configurations, the Python programs, the audit, and much of the prose of this paper were produced with the assistance of large language models, under the direction and review of the author. We record this not as a disclosure ritual but because the experience bears on a question of method, and the question is more interesting than it first appears.

An LLM is a statistical predictor of text. It is, by construction, the philosophical opposite of the idea behind TLA+: where a specification derives its value from meaning exactly what it says, a language model produces what is likely to be said. One might therefore expect the two to mix badly, and in one respect they do. The $\max(D_P, D_A)$ claim of Section 7 is precisely the kind of error a statistical writer makes, since it is plausible, it is symmetric, it resembles bounds that appear in many published protocols, and it survived an exhaustive model-checking pass because the configurations that had been written happened to be the ones where it coincides with the truth. Nothing about fluent text signals whether the bound it states is a theorem or a habit.

What makes the combination work is that nothing generated had to be trusted. A TLA+ module either parses or does not, an invariant either survives an exhaustive search or yields a trace, a TLAPS obligation is either proved or not, and a weakened variant either produces its advertised

violation or fails its regression target. Everything an LLM produced for this paper landed in a world with a referee, and the referee is deterministic. The division of labor is then natural. Generation is fast and fallible, checking is mechanical and reliable, and judgment (which questions to ask, which configurations distinguish hypotheses, which claims to publish) remains with the author. The max error and its correction, both produced by the same process, are the demonstration: the error came from fluent generation, and the correction came from a distinguishing configuration whose verdict did not depend on anyone’s fluency.

The practical observation is that this changes who uses formal methods, and in a direction that we did not expect. Engineers do not avoid TLA+ because they doubt its value; they avoid it because writing and maintaining a specification, its configurations, its counterexamples, and its harness has been laborious, and the labor recurs with every protocol change. In this project the repository, comprising roughly a dozen TLC configurations, two weakened variants with mechanical diff-checking, a TLAPS module, two Python reference models, and a recorded verification pipeline, was built and repeatedly revised in days rather than months, and each revision was quick enough that it was never tempting to skip the re-check. A statistical text predictor, the least trustworthy component imaginable by the standards of formal methods, thus becomes the reason more code gets model checked. We find the same effect in the audit of Section 11: reading two fifteen-year-old codebases against a checklist is exactly the tedious, mechanical, judgment-light work that assistance makes routine, and it surfaced an undocumented safety-critical inequality in an evening.

Two cautions temper the optimism. The first we have already stated: generated claims that are not machine-checkable, such as prose arguments and quantitative statements about code, inherit no guarantee from the checked parts, and the reader should weigh them as they would weigh any unverified claim. The second is that a model checker answers only the configurations it is given, and an LLM is happy to generate configurations that confirm rather than distinguish. The discipline that caught the max error, writing configurations whose purpose is to separate competing hypotheses, is a human discipline, and nothing in the tooling enforces it.

References

- [1] M. Trencsényi, A. Gazsó, and H. Reinhardt. “PaxosLease: Diskless Paxos for Leases.” arXiv:1209.4187, 2012. <https://arxiv.org/abs/1209.4187>
- [2] L. Lamport. “Paxos Made Simple.” *ACM SIGACT News*, 32(4), 2001.
- [3] C. Gray and D. Cheriton. “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency.” *SOSP*, 1989.
- [4] D. Ongaro and J. Ousterhout. “In Search of an Understandable Consensus Algorithm.” *USENIX ATC*, 2014.
- [5] S. Chand, Y. Liu, and S. Stoller. “Formal Verification of Multi-Paxos for Distributed Consensus.” *FM*, 2016.
- [6] M. Burrows. “The Chubby Lock Service for Loosely-Coupled Distributed Systems.” *OSDI*, 2006.
- [7] T. Chandra, R. Griesemer, and J. Redstone. “Paxos Made Live: An Engineering Perspective.” *PODC*, 2007.
- [8] F. Hupfeld, B. Kolbeck, J. Stender, M. Höggqvist, T. Cortes, J. Malo, and J. Marti. “FaTLease: Scalable Fault-Tolerant Lease Negotiation with Paxos.” *HPDC*, 2008.
- [9] L. Lamport. “The Part-Time Parliament.” *ACM Transactions on Computer Systems*, 16(2), 1998.
- [10] B. Kolbeck, M. Höggqvist, J. Stender, and F. Hupfeld. “Flease: Lease Coordination without a Lock Server.” *IPDPS*, 2011.
- [11] U. Sharma, R. Jung, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich. “Grove: a Separation-Logic Library for Verifying Distributed Systems.” *SOSP*, 2023.
- [12] I. Moraru, D. G. Andersen, and M. Kaminsky. “Paxos Quorum Leases: Fast Reads Without Sacrificing Writes.” *SoCC*, 2014.
- [13] H. Cirstea, M. A. Kuppe, B. Loillier, and S. Merz. “Validating Traces of Distributed Programs Against TLA+ Specifications.” arXiv:2404.16075, 2024.